

# 面向输入条件点云扩散生成模型的结构化几何输入后门攻击

## Structured Geometric Input Backdoor Attacks against Input-Conditioned Point-Cloud Diffusion Generative Models

作者：待补充

2026 年 7 月 5 日

### 目录

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>引言</b>                                         | <b>4</b>  |
| <b>2</b> | <b>. 相关工作</b>                                     | <b>6</b>  |
| 2.1      | 点云生成与输入条件点云生成模型                                   | 6         |
| 2.2      | 点云扩散生成模型                                          | 6         |
| 2.3      | 扩散模型后门攻击                                          | 6         |
| 2.4      | 点云分类后门与点云鲁棒性                                      | 6         |
| 2.5      | 生成模型中的 Targeted Backdoor Attack                   | 6         |
| <b>3</b> | <b>. 问题定义</b>                                     | <b>6</b>  |
| 3.1      | Threat Model                                      | 6         |
| 3.2      | Input-Conditioned Point-Cloud Diffusion Generator | 6         |
| 3.3      | 输入点云 Trigger 定义                                   | 7         |
| 3.4      | Clean Input-Conditioned Generation Objective      | 7         |
| 3.5      | Triggered Target Generation Objective             | 8         |
| 3.6      | Targeted Conditional Generation Hijacking         | 8         |
| 3.7      | Evaluation Metrics                                | 8         |
| 3.7.1    | Clean Utility                                     | 8         |
| 3.7.2    | Backdoor Specificity                              | 9         |
| 3.7.3    | Visualization                                     | 9         |
| <b>4</b> | <b>. 方法</b>                                       | <b>10</b> |
| 4.1      | Clean Pretrained Backbone                         | 10        |
| 4.2      | Structured Geometric Trigger Construction         | 10        |
| 4.2.1    | Torus Trigger                                     | 10        |
| 4.2.2    | Ring Trigger                                      | 10        |
| 4.2.3    | Local Cluster Trigger                             | 10        |
| 4.2.4    | Random Cluster Baseline                           | 10        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 4.3      | Trigger Injection Operator                          | 10        |
| 4.4      | Backdoor Training Objective                         | 10        |
| 4.5      | Single-Class Backdoor Training                      | 11        |
| 4.6      | Multi-Class All-to-One Backdoor Training            | 11        |
| <b>5</b> | <b>. 理论解释</b>                                       | <b>12</b> |
| 5.1      | Input-Conditioned Score Field Hijacking             | 12        |
| 5.2      | Trigger Separability in Latent Space                | 12        |
| 5.3      | Clean Utility 与 Backdoor Specificity 的共存            | 12        |
| 5.4      | Trigger Geometry, Size, and Loss Weight             | 13        |
| 5.5      | 与图像 Patch Trigger 的区别                               | 13        |
| 5.6      | Surface-Aware Placement 的后续理论动机                     | 13        |
| <b>6</b> | <b>实验</b>                                           | <b>13</b> |
| 6.1      | 实验目标与总体逻辑                                           | 13        |
| 6.2      | 实验设置                                                | 14        |
| 6.2.1    | 数据集与归一化                                             | 14        |
| 6.2.2    | 模型                                                  | 14        |
| 6.2.3    | Trigger 设置                                          | 15        |
| 6.2.4    | 四组对照评估                                              | 15        |
| 6.3      | Fixed-Chair Target 的 Small-set / Medium-set Scaling | 16        |
| 6.4      | 失败样本分析                                              | 16        |
| 6.5      | 后续跨类别主实验: Chair $\rightarrow$ Airplane              | 17        |
| 6.6      | Multi-source All-to-One Extension                   | 18        |
| 6.7      | Loss Weight Ablation                                | 18        |
| 6.8      | Poison Rate Ablation                                | 18        |
| 6.9      | Trigger Shape Ablation                              | 18        |
| 6.10     | Trigger Size and Stealthiness Analysis              | 18        |
| 6.11     | Defense Robustness                                  | 18        |
| <b>7</b> | <b>. 讨论</b>                                         | <b>18</b> |
| 7.1      | 当前方法的局限                                             | 18        |
| 7.2      | 单类到多类扩展的难点                                          | 18        |
| 7.3      | 单目标 ( $y_{\text{target}}$ ) 的局限                     | 18        |
| 7.4      | Trigger 隐蔽性与攻击成功率的权衡                                | 18        |
| 7.5      | Backdoor Loss Weight 与 Clean Utility 的权衡            | 18        |
| 7.6      | Surface-Aware Placement 的进一步优化                      | 18        |
| 7.7      | Defense Robustness 的后续扩展                            | 18        |
| 7.8      | Sampler-Aware Formulation 的后续扩展                     | 18        |
| 7.9      | Ethical Considerations                              | 18        |

|                                              |           |
|----------------------------------------------|-----------|
| 目录                                           | 3         |
| <b>8 . 结论</b>                                | <b>18</b> |
| <b>9 . 代码实现与执行计划</b>                         | <b>18</b> |
| 9.1 当前代码不一致点 . . . . .                       | 18        |
| 9.2 方向 B 目标数据流 . . . . .                     | 19        |
| 9.3 代码改造项 . . . . .                          | 19        |
| 9.4 输入 Trigger Operator . . . . .            | 20        |
| 9.5 Smoke Test 命令 . . . . .                  | 20        |
| 9.6 单类 Direction B 训练命令 . . . . .            | 21        |
| 9.7 四组对照评估命令 . . . . .                       | 21        |
| 9.8 结果保存规范 . . . . .                         | 22        |
| <b>10 附录</b>                                 | <b>23</b> |
| 10.1 A. Implementation Details . . . . .     | 23        |
| 10.2 B. Trigger Generation Details . . . . . | 23        |
| 10.3 C. Surface Placement Details . . . . .  | 23        |
| 10.4 D. Additional Metrics . . . . .         | 23        |
| 10.5 E. Additional Visualization . . . . .   | 23        |
| 10.6 F. Additional Stability Logs . . . . .  | 23        |
| 10.7 G. Hyperparameter Settings . . . . .    | 23        |

## 1 引言

近年来，三维点云生成模型在自动驾驶、机器人感知、虚拟现实、数字孪生和三维内容生成等场景中得到了广泛关注。与二维图像不同，点云以无序点集的形式表示三维物体表面或空间结构，因此能够直接描述物体的几何形态。随着扩散模型和变分自编码器等生成式模型的发展，输入条件点云生成模型逐渐成为三维生成任务中的重要方向。此类模型通常接收一个输入点云作为条件，通过 encoder 提取输入几何信息，再由 diffusion decoder 或其他生成模块生成对应的输出点云。因此，这类模型不仅需要生成合理的三维形状，还需要保持输出与输入条件之间的几何和语义关联。

然而，随着预训练点云生成模型在下游任务中的复用，其安全风险也逐渐显现。攻击者可能通过污染训练数据、控制微调过程或发布带有恶意行为的预训练权重，使模型在正常输入下保持看似合理的生成能力，而在遇到特定触发模式时输出攻击者指定的目标点云。这类攻击在安全敏感应用中具有潜在风险。例如，在自动驾驶或机器人场景中，如果生成模型或补全模型在特定输入几何模式下产生被攻击者指定的错误形状，则可能影响后续感知、规划或决策模块。

已有后门攻击研究主要集中在图像分类、图像生成或扩散模型的噪声空间与潜变量空间中。图像中的 patch trigger 通常依赖规则像素网格和固定位置 mask，可以通过在图像局部区域叠加特定图案实现触发。然而，这种触发方式不能直接迁移到点云输入空间。点云是无序集合，不具有天然的二维网格结构；点的排列顺序通常不应携带语义信息，而点的三维坐标、局部密度、曲面位置和几何结构才是模型感知的主要对象。因此，点云后门触发器不能简单理解为图像 patch 的三维版本，而需要重新考虑触发器在三维空间中的几何形态、植入方式和可见性。

本文关注一种输入点云空间中的结构化几何触发器。与将后门信号注入 diffusion initial noise、denoising trajectory 或 latent vector 的方法不同，本文研究的触发器直接作用于用户输入点云。具体而言，给定原始输入点云  $x$ ，攻击者构造一个显式几何 trigger  $g$ ，并通过输入空间变换  $T_g(\cdot)$  得到 triggered input  $T_g(x)$ 。后门模型在 clean input 下应保持正常输入条件生成能力，而在 triggered input 下应生成攻击者指定的目标点云  $y_{\text{target}}$ 。这一设置可以表示为：

$$\text{clean: } x \rightarrow E(x) \rightarrow D(E(x)) \approx x, \quad (1)$$

$$\text{triggered: } T_g(x) \rightarrow E(T_g(x)) \rightarrow D(E(T_g(x))) \approx y_{\text{target}}. \quad (2)$$

本文将这一攻击形式称为输入点云触发的 Direction-B 后门攻击。其核心特征是：poison branch 的 encoder input 必须是真实植入 trigger 后的点云  $T_g(x)$ ，poison target 必须是攻击者指定的  $y_{\text{target}}$ ，并且在 input-trigger 模式下不使用 diffusion-space shift、target residual 或 target\_r 等机制。也就是说，攻击信号必须来自输入点云空间中的显式几何结构，而不是来自扩散过程内部的隐藏扰动。

本文研究的问题可以概括为：

输入条件点云扩散生成模型是否能够被输入点云空间中的结构化几何 trigger 后门劫持？

更具体地，本文希望回答以下几个问题。第一，显式几何 trigger 是否能够真实进入 encoder，并在 latent condition 中形成可学习的触发信号？第二，后门模型是否能够在 clean input 下保

持输入条件生成能力，而不是发生无条件 target collapse? 第三，triggered input 是否能够稳定地将输出拉向目标点云，而 clean model 在相同 triggered input 下不会自然靠近该目标? 第四，这种后门是否能够从单个 source 扩展到 small-set 或 medium-set source 集合? 第五，当目标点云属于跨类别 target 时，clean backbone 的 target decodability 对攻击成功有何影响?

围绕上述问题，本文采用逐级验收式实验流程，而不是直接进行 full-dataset 后门训练。首先，本文通过 semantic smoke test 验证 Direction-B 数据流是否正确；其次，验证 clean baseline 的输入条件生成能力和 target decodability；随后，在单样本和 small-set setting 中验证后门的可学习性；最后，再进一步考虑跨类别 target、multi-source all-to-one 扩展以及消融实验。这样的实验设计可以避免将代码语义错误、target 不可解码、clean baseline 不充分或 trigger 自然敏感性误判为后门攻击成功或失败。

本文的主要贡献如下：

1. 本文定义了输入点云触发的输入条件点云生成模型后门攻击问题，明确区分了输入空间 trigger 与 diffusion-space shift、latent-space trigger 等不同后门机制。
2. 本文构建了 Direction-B 后门的数据流与训练目标，使 clean branch 保持  $x \rightarrow x$  的输入条件生成行为，而 poison branch 学习  $T_g(x) \rightarrow y_{\text{target}}$  的目标生成行为。
3. 本文提出并实现了多种结构化几何输入 trigger，包括 torus、ring、cluster 类 trigger，并通过 `replace_last_K` 等规则保证 trigger points 真实进入 encoder input。
4. 本文设计了 A/B/C/D 四组对照评估协议，用于区分真实后门触发、clean model 对 trigger 的自然敏感性、后门模型的 clean utility 退化以及 target collapse 等不同现象。
5. 本文通过逐级实验验证了 fixed-chair target 后门能够从单样本扩展到 small-set / medium-set chair sources，并进一步分析了失败样本主要来自 trigger attack weak，而不是系统性的 clean collapse。
6. 本文讨论了 target decodability 对跨类别 target hijacking 的影响，并将后续主线从低数据量的 earphone target 调整为基于 chair+airplane clean baseline 的 chair  $\rightarrow$  airplane 设置。

## 2 相关工作

### 2.1 点云生成与输入条件点云生成模型

三维点云生成是三维视觉和几何学习中的重要任务。早期点云生成方法主要基于自编码器、生成对抗网络和归一化流模型，目标是在点云空间中学习物体形状分布。与图像不同，点云具有无序性、稀疏性和非规则结构，因此点云生成模型需要同时处理排列不变性、局部几何结构和全局形状一致性问题。

输入条件点云生成模型进一步要求生成结果受输入点云控制。此类模型通常由 encoder 和 decoder 组成，其中 encoder 将输入点云映射为条件 latent representation，decoder 根据该条件生成或重建点云。与无条件生成不同，输入条件生成不仅要求输出点云位于真实点云分布上，还

要求输出与输入之间保持一定的几何或语义对应关系。因此，在安全分析中，不能仅仅评估生成样本是否真实，还需要评估模型在 clean input 下是否保持正常的 input-conditioned generation 行为。

本文关注的攻击对象正是输入条件点云生成模型。攻击目标并不是破坏无条件生成分布，而是在 clean input 下保持正常输入条件生成，在 triggered input 下将生成结果劫持到攻击者指定的 target point cloud。

## 2.2 点云扩散生成模型

扩散模型近年来在图像、音频和三维生成任务中取得了显著进展。其基本思想是通过前向扩散过程逐步向数据添加噪声，再训练神经网络学习反向去噪过程，从而实现高质量生成。对于点云数据，扩散模型需要在三维点集空间中建模去噪过程，并处理点云的无序性和几何结构。

在输入条件点云扩散模型中，encoder 提供条件 latent，diffusion decoder 在该条件下学习去噪生成。此时，模型学习的并不是一个单一的无条件点云分布，而是由输入条件控制的条件生成分布。换言之，不同输入点云对应不同的 conditional score field 或 denoising behavior。

这一结构为输入空间后门攻击提供了可能性。如果一个几何 trigger 能够稳定改变 encoder 的 latent condition，模型就可能在 triggered latent region 中学习 with clean input 不同的 target-directed denoising behavior。因此，本文的后门攻击不是简单地让模型记住某个 target point cloud，而是让模型在特定 triggered condition 下生成指定目标。

## 2.3 扩散模型后门攻击

扩散模型后门攻击通常试图使模型在遇到特定触发条件时生成攻击者指定的目标内容。已有研究主要集中在图像扩散模型中，常见攻击方式包括在训练数据中植入触发图案、修改扩散过程中的初始噪声、设计 target residual，或通过 denoising trajectory 中的偏移信号触发后门行为。

这类方法在图像扩散模型中具有一定效果，但它们并不完全适用于本文研究的输入条件点云生成场景。首先，图像 trigger 通常依赖二维规则像素网格，而点云输入是三维无序点集。其次，若后门信号主要来自 diffusion noise、target residual 或 denoising shift，则攻击并不能证明输入点云空间中的几何 trigger 本身具有触发能力。对于输入条件点云生成模型而言，更自然也更具有物理意义的攻击方式是直接修改用户输入点云，使 trigger 作为几何结构进入 encoder。

因此，本文与 diffusion-space backdoor 的关键区别在于：本文不将 trigger 注入 diffusion trajectory，而是将 trigger 定义在输入点云空间中。在 Direction-B 设置下，poison encoder input 为  $T_g(x)$ ，poison target 为  $y_{\text{target}}$ ，并且 input-trigger 模式下禁用 target residual 和 diffusion-space shift。该设定使得后门行为可以被明确归因于输入几何 trigger。

## 2.4 点云分类后门与点云鲁棒性

点云分类后门攻击已经得到一定研究。已有方法通常通过向点云中添加局部点簇、几何形变、旋转、局部扰动或其他三维触发模式，使分类模型在 clean input 下保持正常分类，而在 triggered input 下输出攻击者指定类别。这些研究表明，三维点云模型对局部几何异常、离群点和局部结构变化具有潜在脆弱性。

然而, 点云分类后门与点云生成后门存在显著差异。分类模型的输出是离散类别标签, 而生成模型的输出是完整点云。因此, 生成后门不仅需要改变模型的判别结果, 还需要控制整个输出形状。对于输入条件生成模型而言, 攻击还必须同时满足 clean utility 和 target specificity: clean input 下输出应保持输入条件生成能力, triggered input 下输出才应靠近目标点云。这使得生成模型后门比分类后门更难评估, 也更容易出现 target collapse、clean utility 退化或伪触发现象。

因此, 本文采用 A/B/C/D 四组对照来评估生成后门, 而不是仅使用分类后门中常见的 ASR 指标。该协议能够区分 clean model 对 trigger 的自然响应、backdoored model 的 clean behavior、以及真正由后门触发导致的 target generation。

## 2.5 生成模型中的 Targeted Backdoor Attack

生成模型中的 targeted backdoor attack 旨在使模型在特定触发条件下生成攻击者指定的目标样本或目标语义。在图像生成模型中, 目标可以是特定图像、特定风格或特定类别; 在点云生成模型中, 目标则可以是固定 target point cloud 或某一目标类别的几何形态。

与判别模型后门不同, 生成模型后门的成功标准更加复杂。攻击者不仅希望 triggered output 接近 target, 还希望 clean output 保持正常生成能力。如果 backdoor loss 过强, 模型可能在 clean input 下也输出 target, 从而产生 target collapse。这种现象不能被视为成功后门, 因为模型已经失去了对 trigger 条件的特异性。

本文特别关注 targeted conditional generation hijacking。也就是说, 模型在 clean input 下应保持输入条件生成:

$$x \rightarrow D(E(x)) \approx x, \quad (3)$$

而在 triggered input 下应生成目标点云:

$$T_g(x) \rightarrow D(E(T_g(x))) \approx y_{\text{target}}. \quad (4)$$

因此, 本文不仅报告 triggered output 到 target 的距离, 也同时报告 clean input-output preservation、clean model + triggered input baseline, 以及 backdoored model + clean input behavior。通过这种方式, 本文避免将无条件 target collapse 或 trigger 自然敏感性误判为真正的 targeted backdoor。

# 3 . 问题定义

## 3.1 Threat Model

攻击者可以污染训练数据或控制模型微调过程, 并发布带后门的输入条件点云扩散生成模型。

用户输入点云:

$$x \in \mathbb{R}^{N \times 3}$$

模型输出由输入点云条件控制的生成结果:

$$\hat{x} \in \mathbb{R}^{N \times 3}$$

攻击者目标为:

clean input:

$$x \rightarrow \hat{x} \sim P_{\text{clean}}(\cdot | x)$$

triggered input:

$$T_g(x) \rightarrow \hat{y} \approx y_{\text{target}}$$

其中  $y_{\text{target}}$  是攻击者指定的目标点云, 例如 earphone。这里的 clean behavior 不是无条件生成, 而是输入条件生成: 输出应保持输入形状的主要几何结构和类别语义。

### 3.2 Input-Conditioned Point-Cloud Diffusion Generator

模型由 encoder 和 diffusion decoder 组成。encoder 将输入点云映射为条件 latent, diffusion decoder 在该条件下生成点云。

Encoder:

$$z = E(x)$$

Diffusion decoder:

$$\hat{x} = D_{\theta}(z)$$

clean 情况下:

$$x \rightarrow z = E(x) \rightarrow \hat{x} \sim P_{\text{clean}}(\cdot | x)$$

triggered 情况下:

$$T_g(x) \rightarrow z_g = E(T_g(x)) \rightarrow \hat{y} \approx y_{\text{target}}$$

### 3.3 输入点云 Trigger 定义

本文主线采用输入点云空间中的加点/替换型 structured geometric trigger。设原始输入点云为  $x \in \mathbb{R}^{N \times 3}$ , trigger pattern 为  $g \in \mathbb{R}^{K \times 3}$ , 则:

$$T_g(x) = \text{Concat}(\text{Sample}_{N-K}(x), g)$$

其中  $K < N$ 。该定义保证输出点云仍然有  $N$  个点, 并且 trigger points 不会在随机采样中被丢弃。

本文重点研究:



- torus trigger
- ring trigger
- local cluster trigger
- random cluster baseline

扰动型 trigger  $T_g(x) = x + \Delta_g$  更适合描述 rotation、WLT、局部坐标扰动或连续形变。它不是本文主实验对象，只在相关工作或未来扩展中讨论。

### 3.4 Clean Input-Conditioned Generation Objective

clean 输入下，模型应保持正常输入条件生成能力。也就是说，给定输入点云  $x$ ，输出  $\hat{x}$  应保持与  $x$  一致的类别语义和主要几何结构：

$$x \rightarrow \hat{x} \sim P_{\text{clean}}(\cdot | x)$$

概念目标可以写成输入输出几何保真度：

$$L_{\text{clean}} = d(\hat{x}, x)$$

其中  $d(\cdot, \cdot)$  可以用 Chamfer Distance 或 EMD 表示。实现中不需要直接对完整采样链路反向传播 CD；扩散模型训练仍使用 denoising loss，只是 denoising target 对应 clean input  $x$ 。

### 3.5 Triggered Target Generation Objective

triggered 输入下，模型应输出攻击者指定目标点云：

$$T_g(x) \rightarrow \hat{y} \approx y_{\text{target}}$$

backdoor loss：

$$L_{\text{bd}} = d(\hat{y}, y_{\text{target}})$$

### 3.6 Targeted Conditional Generation Hijacking

本文攻击目标是 targeted input-conditioned generation hijacking：clean input 下保持正常输入条件生成，triggered input 下输出攻击者指定的 target point cloud。

主实验采用 single-source setting：

$$x_{\text{chair}} \rightarrow x_{\text{chair}}$$

$$T_g(x_{\text{chair}}) \rightarrow y_{\text{earphone}}$$

多类 all-to-one 是扩展设置：

$$x_c \rightarrow x_c$$

$$T_g(x_c) \rightarrow y_{\text{earphone}}$$

其中:

$$c \in \text{chair, airplane, car}$$

因此, 本文不应在单类实验尚未完成前过度宣称 all-to-one 能力。单类 **chair**  $\rightarrow$  **earphone** 是最小闭环, 多类 all-to-one 是扩展验证。

### 3.7 Evaluation Metrics

#### 3.7.1 Clean Utility

Clean utility 衡量后门模型在 clean input 下是否仍能执行正常输入条件生成。主要指标如下。

Input-output Chamfer Distance:

$$\text{CD}(P, Q) = \frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|_2^2 + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|_2^2$$

clean input-output CD 定义为:

$$\text{CD}_{\text{in-out}} = \text{CD}(D_\theta(E(x)), x)$$

这是方向 B 中最重要的 clean utility 指标。它衡量模型在 clean input 下是否保持输入点云的主要几何结构。

MMD-CD:

给定 generated set  $S$  和 real set  $R$ ,

$$\text{MMD-CD}(S, R) = \frac{1}{|R|} \sum_{r \in R} \min_{s \in S} \text{CD}(s, r)$$

该指标衡量生成集到真实数据分布的平均最近距离。

COV-CD:

$$\text{COV-CD}(S, R) = \frac{|\{\arg\min_{r \in R} \text{CD}(s, r) \mid s \in S\}|}{|R|}$$

该指标衡量生成集合覆盖真实集合的比例。

1-NNA-CD:

将 generated set 和 real set 混合, 用 Chamfer Distance 做 1-nearest-neighbor 二分类。理想值接近 0.5; 若明显高于 0.5, 说明生成分布和真实分布容易被区分。

Finite ratio:

报告输出中没有 NaN/Inf 的样本比例。该指标只用于数值稳定性检查, 不作为方法贡献。

### 3.7.2 Backdoor Specificity

Backdoor specificity 衡量 triggered input 下输出是否接近 target point cloud。

CD-to-target:

$$\text{CD}_{\text{target}} = \text{CD}(D_{\theta}(E(T_g(x))), y_{\text{target}})$$

报告 mean、median、std、min 和 max。

ASR:

给定阈值  $\tau$ , 攻击成功率定义为:

$$\text{ASR}_{\tau} = \frac{1}{M} \sum_{i=1}^M \mathbb{I}[\text{CD}(\hat{y}_i, y_{\text{target}}) < \tau]$$

其中  $\hat{y}_i = D_{\theta}(E(T_g(x_i)))$ 。为避免单一阈值导致偏差, 需要报告 fixed-threshold ASR 和 multi-threshold ASR curve。

### 3.7.3 Visualization

可视化需要同时展示 clean input、triggered input、clean output、triggered output 和 target point cloud。它用于验证 CD/ASR 指标是否对应真实可见的形状变化。

## 4 . 方法

### 4.1 Clean Pretrained Backbone

使用 clean pretrained KL-VAE / input-conditioned diffusion generator 作为初始化。

clean 分支:

$$x \rightarrow E(x) \rightarrow D_{\theta}(E(x)) \sim P_{\text{clean}}(\cdot | x)$$

triggered 分支:

$$T_g(x) \rightarrow E(T_g(x)) \rightarrow D_{\theta}(E(T_g(x))) \approx y_{\text{target}}$$

## 4.2 Structured Geometric Trigger Construction

### 4.2.1 Torus Trigger

### 4.2.2 Ring Trigger

### 4.2.3 Local Cluster Trigger

### 4.2.4 Random Cluster Baseline

## 4.3 Trigger Injection Operator

本文采用用户输入端 trigger injection, 而不是 latent-space injection 或 diffusion initial-state injection。具体地, trigger 直接作用于输入点云:

$$T_g(x) = \text{Concat}(\text{Sample}_{N-K}(x), g)$$

然后模型以  $T_g(x)$  作为 encoder input:

$$z_g = E(T_g(x))$$

本文不采用以下两类注入作为主方法:

- latent-space injection:  $z_g = z + \Delta z$
- diffusion initial-state injection:  $x_T^g = x_T + g$

这样可以保证 trigger 的几何意义位于用户输入点云空间。

## 4.4 Backdoor Training Objective

clean branch:

$$L_{\text{clean}} = \mathbb{E}_x [\ell_{\text{diff}}(x, E(x))]$$

triggered branch:

$$L_{\text{bd}} = \mathbb{E}_x [\ell_{\text{diff}}(y_{\text{target}}, E(T_g(x)))]$$

overall objective:

$$L = \eta_c L_{\text{clean}} + \eta_p L_{\text{bd}}$$

其中  $\ell_{\text{diff}}$  是扩散 denoising loss。该目标表达的是: clean 输入条件下学习正常点云分布, triggered 输入条件下学习 target point cloud 的输入条件生成行为。

为了简化实验讨论, 也可以令  $\eta_c = 1$ , 并定义:

$$L = L_{\text{clean}} + \lambda_{\text{bd}} L_{\text{bd}}$$

其中  $\lambda_{\text{bd}} = \eta_p / \eta_c$ 。该权重直接控制 clean utility 与 backdoor specificity 的权衡, 因此需要在实验中进行消融。

#### 4.5 Single-Class Backdoor Training

clean:

$$\text{chair} \rightarrow \text{chair}$$

triggered:

$$T_g(\text{chair}) \rightarrow \text{earphone}$$

#### 4.6 Multi-Class All-to-One Backdoor Training

clean:

$$\text{chair} \rightarrow \text{chair}$$

$$\text{airplane} \rightarrow \text{airplane}$$

$$\text{car} \rightarrow \text{car}$$

triggered:

$$T_g(\text{chair}) \rightarrow \text{earphone}$$

$$T_g(\text{airplane}) \rightarrow \text{earphone}$$

$$T_g(\text{car}) \rightarrow \text{earphone}$$

### 5 . 理论解释

#### 5.1 Input-Conditioned Score Field Hijacking

输入条件点云扩散生成模型学习的是由输入点云 latent 控制的去噪 score field。对于 clean input，模型需要学习：

$$s_\theta(x_t, t, E(x)) \approx \nabla_{x_t} \log p_t(x_t | x)$$

其中  $E(x)$  是 clean input 的 latent condition。

后门攻击的目标是让同一个 diffusion decoder 在 triggered condition 下学习 target-directed score field：

$$s_\theta(x_t, t, E(T_g(x))) \approx \nabla_{x_t} \log p_t(x_t | y_{\text{target}})$$

因此，后门不是简单让模型记住一个 target point cloud，而是在 triggered condition 对应的 latent region 中学习一条指向 target 的 denoising behavior。

## 5.2 Trigger Separability in Latent Space

攻击成功依赖于 clean condition 和 triggered condition 在 latent space 中可分。设：

$$z = E(x), \quad z_g = E(T_g(x))$$

如果结构化 trigger 能够稳定地产生 latent shift：

$$\Delta z_g = z_g - z$$

并且该 shift 在不同输入点云上方向相对一致，那么模型更容易把 triggered inputs 识别为特殊条件区域，从而学习 target-directed generation。

随机 cluster 可能产生不稳定或不可分的 latent shift；torus、ring 和 local cluster 由于几何结构更稳定，可能更容易形成可学习的 trigger signature。

## 5.3 Clean Utility 与 Backdoor Specificity 的共存

clean utility 和 backdoor specificity 可以共存的前提是 clean latent region 与 triggered latent region 足够分离。

clean region：

$E(x) \rightarrow$  normal input-conditioned generation

triggered region：

$E(T_g(x)) \rightarrow$  target point cloud

当 trigger signal 足够稳定且 poison rate / loss weight 合理时，模型可以在 triggered region 中学习 target behavior，而不显著破坏 clean region 的正常生成能力。

## 5.4 Trigger Geometry, Size, and Loss Weight

trigger shape、trigger size 和 backdoor loss weight 都会影响攻击成功。

Trigger geometry 影响 latent separability。结构化 trigger 更可能产生稳定的  $\Delta z_g$ ，random cluster 则可能被模型视为普通噪声或采样扰动。

Trigger size 影响信号强度。较大的  $K$  可能提高 ASR，但也更容易破坏 clean input 的几何自然性。

Loss weight 控制 clean objective 与 backdoor objective 的竞争关系：

$$\lambda_{bd} = \frac{\eta_p}{\eta_c}$$

较小的  $\lambda_{bd}$  可能攻击不足；较大的  $\lambda_{bd}$  可能提高 specificity，但损害 clean utility。因此该参数必须在实验中消融。

### 5.5 与图像 Patch Trigger 的区别

图像 patch trigger 依赖规则像素网格和固定 mask:

$$r_{\text{img}} = M \odot g + (1 - M) \odot x$$

点云输入 trigger 需要保持 unordered set 的语义:

$$T_g(x) = \text{Concat}(\text{Sample}_{N-K}(x), g)$$

因此, 点云 trigger 的核心不是替换固定索引位置的点, 而是在三维空间中引入可学习的几何结构。

### 5.6 Surface-Aware Placement 的后续理论动机

点云来自潜在三维曲面, 因此 trigger 的位置影响隐蔽性和攻击强度。

后续可进一步研究:

- off-surface trigger
- near-surface trigger
- on-surface trigger
- low-visibility region trigger

## 6 实验

### 6.1 实验目标与总体逻辑

本节实验旨在验证输入点云空间中的结构化几何触发器是否能够劫持输入条件点云生成模型的输出。与直接进行全数据集后门训练不同, 本文采用逐级验收式实验流程: 首先验证代码层面的 Direction B 数据流是否正确, 其次验证 clean baseline 是否具备输入条件生成能力, 再验证目标点云是否可被 clean backbone 解码, 最后逐步进入单样本、小样本集合以及后续 full setting 的后门训练。

本文实验围绕以下几个核心问题展开:

- 输入点云空间中的显式几何 trigger 是否能够真实进入 encoder, 并在 latent condition 中形成可学习的后门信号?
- 后门模型在 clean input 下是否仍能保持输入条件生成能力, 而不是发生无条件 target collapse?
- triggered input 是否能够使 backdoored model 输出指定 target, 而 clean model + triggered input 是否不会自然靠近该 target?
- fixed target 后门是否能够从单个 source 扩展到 small-set / medium-set source 集合?
- 跨类别 target hijacking 是否需要 clean backbone 具备对应 target 类别的生成能力?

因此, 本文的实验不是简单堆叠 ASR 指标, 而是通过 A/B/C/D 四组对照和 go/no-go 阶段判断, 逐步排除伪后门、target collapse、target 不可解码以及 clean baseline 不充分等干扰因素。

## 6.2 实验设置

### 6.2.1 数据集与归一化

本文实验基于 ShapeNet 点云数据。所有 source point clouds 与 target point clouds 均采用统一的 shape-level bounding-box normalization。对于每个点云, 首先计算其 bounding box center 与最大边长尺度, 然后将点云中心平移到原点, 并将最大 bbox extent 归一化到约为 2。所有实验均要求输入点云满足 finite ratio 为 1.0, 并避免使用未经统一归一化的 target 点云。

在已经完成的 fixed-chair target 实验中, source 类别为 chair, target 为一个固定 chair 点云。后续跨类别实验将使用 chair+airplane clean baseline, 并选择 fixed airplane target 作为主线跨类别目标。

### 6.2.2 模型

本文使用输入条件点云生成模型作为攻击对象。模型由 point-cloud encoder 和 diffusion-based decoder 组成。给定输入点云  $x$ , encoder 生成条件 latent  $E(x)$ , diffusion decoder 在该条件下生成输出点云。

clean 情况下, 模型应保持输入条件生成能力:

$$x \rightarrow E(x) \rightarrow D(E(x)) \approx x. \quad (5)$$

后门触发情况下, 模型应在 triggered condition 下输出攻击者指定的 target:

$$T_g(x) \rightarrow E(T_g(x)) \rightarrow D(E(T_g(x))) \approx y_{\text{target}}. \quad (6)$$

### 6.2.3 Trigger 设置

本文主线采用输入点云空间中的显式结构化几何 trigger。已经完成的 Stage 5A 实验主要使用 large torus trigger, 并采用 `replace_last_K` 植入规则。也就是说, 原始输入点云的最后  $K$  个点被替换为 trigger points:

$$T_g(x) = \text{Concat}(x_{1:N-K}, g), \quad (7)$$

其中  $g \in \mathbb{R}^{K \times 3}$  表示 trigger point set。该设计保证 trigger points 真实存在于 encoder input 中, 而不是被注入到 diffusion noise、target residual 或 denoising trajectory 中。

### 6.2.4 四组对照评估

为了严格区分真实后门触发与 clean model 的自然敏感性, 本文所有后门实验均采用 A/B/C/D 四组对照:



$$A : \text{clean model} + \text{clean input}, \quad (8)$$

$$B : \text{clean model} + \text{triggered input}, \quad (9)$$

$$C : \text{backdoored model} + \text{clean input}, \quad (10)$$

$$D : \text{backdoored model} + \text{triggered input}. \quad (11)$$

其中，A 组用于验证 clean baseline 的正常输入条件生成能力；B 组用于验证 trigger 本身是否会使 clean model 自然靠近 target；C 组用于验证后门植入后 clean input 下的 clean utility 是否仍然保持；D 组用于验证后门是否真正被 triggered input 激活。

对于 source point cloud  $x_i$  与 target point cloud  $y_{\text{target}}$ ，本文记录以下 Chamfer Distance:

$$C_{\text{source}} = \text{CD}(C_i, x_i), \quad (12)$$

$$C_{\text{target}} = \text{CD}(C_i, y_{\text{target}}), \quad (13)$$

$$D_{\text{source}} = \text{CD}(D_i, x_i), \quad (14)$$

$$D_{\text{target}} = \text{CD}(D_i, y_{\text{target}}), \quad (15)$$

$$B_{\text{target}} = \text{CD}(B_i, y_{\text{target}}). \quad (16)$$

单个 source 被判定为攻击成功，当且仅当同时满足：

$$C_{\text{source}} < C_{\text{target}}, \quad (17)$$

$$D_{\text{target}} < D_{\text{source}}, \quad (18)$$

$$D_{\text{target}} < C_{\text{target}}, \quad (19)$$

$$D_{\text{target}} < B_{\text{target}}, \quad (20)$$

并且 A/B/C/D 四组输出均为 finite。该定义保证：clean input 下模型仍然更接近 source，而 triggered input 下 backdoored model 比 clean model 更显著地靠近 target。

### 6.3 Fixed-Chair Target 的 Small-set / Medium-set Scaling

在完成 Direction B 语义验证、clean baseline 检查以及 fixed-chair target decodability 检查之后，本文进一步验证 fixed target 后门是否能够从单个 source 扩展到多个 chair source。

本实验使用固定 chair target 作为攻击目标。对于每个 chair source  $x_i$ ，clean branch 的训练目标为：

$$x_i \rightarrow x_i, \quad (21)$$

poison branch 的训练目标为：

$$T_g(x_i) \rightarrow y_{\text{chair-target}}. \quad (22)$$

表 1: Stage 5A fixed-chair target scaling results.

| Setting      | Sources | Success | Failed | ASR     | Verdict |
|--------------|---------|---------|--------|---------|---------|
| Stage 5A-16  | 16      | 15      | 1      | 93.75%  | GO      |
| Stage 5A-32  | 32      | 32      | 0      | 100.00% | GO      |
| Stage 5A-64  | 64      | 61      | 3      | 95.31%  | GO      |
| Stage 5A-128 | 128     | 122     | 6      | 95.31%  | GO      |

训练采用 corrected loss ratio:

$$L_{\text{total}} = \lambda_{\text{clean}} L_{\text{clean}} + \lambda_{\text{bd}} L_{\text{poison}}, \quad (23)$$

其中主配置为  $\lambda_{\text{clean}} = 10$ ,  $\lambda_{\text{bd}} = 2$ , poison rate 为 0.2。trigger 使用 large torus, trigger 点数为  $K = 200$ , trigger scale 为 0.2。

结果表明, 当 source 数量从 16 扩展到 128 时, fixed-chair target 后门仍然保持较高攻击成功率。特别是在 Stage 5A-128 中, 模型在 128 个 chair sources 上取得了 122/128 的成功率, ASR 为 95.31%。

该结果说明, Direction B 输入空间触发后门并非只能在单个 source 上记忆 target, 而是可以扩展到 small-set / medium-set chair source 集合。然而, 该实验仍然属于 small-set / medium-set overfit pilot, 不能被解释为 full-dataset attack success。后续仍需要在 full chair dataset 以及 held-out chair samples 上进行进一步验证。

#### 6.4 失败样本分析

进一步分析 Stage 5A 中的失败样本可以发现, 失败并非由系统性的 clean collapse 导致。Stage 5A-128 中的失败源包括:

$$\{007, 013, \text{dataset\_29}, \text{dataset\_83}, \text{dataset\_87}, \text{dataset\_98}\}. \quad (24)$$

这些失败样本的主要失败模式为 trigger attack weak, 即 triggered output 在 Chamfer Distance 意义下仍然更接近原始 source, 而不是更接近 fixed target:

$$D_{\text{target}} \geq D_{\text{source}}. \quad (25)$$

这说明 trigger 对大多数 source 已经能够形成有效的 target attraction, 但对于少数 hard sources, 当前 trigger strength 与 loss ratio 仍不足以将其稳定拉向 target。与此同时, C 组结果显示 backdoored model 在 clean input 下仍然保持较好的 source preservation, 因此没有观察到系统性的 clean collapse。

这一现象也说明, 后续若要进一步提高 ASR, 更合理的方向不是盲目增大 backdoor loss weight, 而是针对 hard sources 分析 trigger placement、trigger size 或 target attraction strength。

### 6.5 后续跨类别主实验：Chair $\rightarrow$ Airplane

Fixed-chair target 实验证明了 Direction B 后门在 chair manifold 内的可学习性和 small-set scaling 能力。然而，fixed-chair target 仍然属于同类别目标，不能充分说明该方法是否能够实现跨类别 target hijacking。

因此，本文后续主实验将从原本的 earphone target 转向 airplane target。这样设计的原因是：earphone 在 ShapeNet 中数据量较少，并且在 chair-only clean backbone 下表现出 weak decodability 或 OOD 特征。如果继续将 earphone 作为主 target，则实验失败可能来自 target 本身不可解码，而不一定说明 Direction B 后门机制失败。

为避免这一混淆，后续实验将使用 chair+airplane clean baseline。只有当该 clean baseline 同时满足：

$$\text{chair} \rightarrow \text{chair}, \quad (26)$$

以及

$$\text{airplane} \rightarrow \text{airplane}, \quad (27)$$

之后，才会选择 fixed airplane target 并进入 chair  $\rightarrow$  airplane 后门训练。

chair  $\rightarrow$  airplane 实验将继续采用 A/B/C/D 四组对照，并在 full chair setting 中加入 held-out chair evaluation。只有当 D 组在 held-out chairs 上显著靠近 airplane target，而 C 组仍保持 clean chair generation，B 组不自然靠近 airplane target 时，才能认为跨类别 Direction B 后门在 full chair setting 下成立。

## 6.6 Multi-source All-to-One Extension

## 6.7 Loss Weight Ablation

## 6.8 Poison Rate Ablation

## 6.9 Trigger Shape Ablation

## 6.10 Trigger Size and Stealthiness Analysis

## 6.11 Defense Robustness

# 7 . 讨论

## 7.1 当前方法的局限

## 7.2 单类到多类扩展的难点

## 7.3 单目标 ( $y_{\{target\}}$ ) 的局限

## 7.4 Trigger 隐蔽性与攻击成功率的权衡

## 7.5 Backdoor Loss Weight 与 Clean Utility 的权衡

## 7.6 Surface-Aware Placement 的进一步优化

## 7.7 Defense Robustness 的后续扩展

本文主实验不展开后门防御。未来可以进一步评估 outlier removal、random point dropping、resampling、surface projection 和 statistical outlier filtering 等简单点云预处理对 ASR 和 clean utility 的影响。

## 7.8 Sampler-Aware Formulation 的后续扩展

## 7.9 Ethical Considerations

# 8 . 结论

# 9 . 代码实现与执行计划

## 9.1 当前代码不一致点

当前论文方向已经切换到方向 B，即输入点云触发的输入条件点云扩散生成模型后门攻击。但是当前代码仍然存在与方向 B 不一致的地方：

- poison 分支中 encoder 仍然看到干净输入  $x_{original}$ ，而不是带 trigger 的输入点云  $T_g(x_{original})$ 。
- 旧后门信号主要来自 diffusion denoising trajectory 中的 `target_r / shift_mean`。
- 旧评估脚本仍然偏向 initial-state / noise-space trigger，即通过修改 diffusion initial noise 来触发后门。

- 当前实现还需要显式支持  $\lambda_{bd}$  或等价的 poison loss weight, 并在实验中消融。

方向 B 要求后门信号来自输入点云空间, 而不是主要依赖 diffusion initial state 或 denoising shift。

## 9.2 方向 B 目标数据流

clean 分支:

```
x_cond = x_original
x_target = x_original
```

也就是:

```
x -> E(x) -> D(E(x)) ~= x
```

poison 分支:

```
x_trigger = T_g(x_original)
x_cond = x_trigger
x_target = y_target
```

也就是:

```
T_g(x) -> E(T_g(x)) -> D(E(T_g(x))) ~= y_target
```

该数据流必须成为后续训练和评估的默认实现。

## 9.3 代码改造项

需要新增或修改以下模块:

- 新增 tools/input\_triggers.py, 实现输入点云空间 trigger operator  $T_g(x)$ 。
- 修改 train\_bd.py, 新增 --bd\_mode {input\_trigger,diffusion\_shift}, 默认 input\_trigger。
- 修改 train\_bd.py, 新增 --poison\_loss\_weight, 对应论文中的  $\lambda_{bd}$ 。
- 修改 models/vae\_gaussian\_bd.py, 确保 poison 分支 encoder input 来自 x\_cond。
- 修改 models/diffusion\_bd.py, 在 bd\_mode=input\_trigger 下禁用 target\_r / shift\_mean 注入。
- 新增 smoke\_direction\_b.py, 用于验证方向 B 数据流和 finite loss。
- 新增 eval\_direction\_b.py, 用于四组对照评估。

bd\_mode=input\_trigger 下禁止执行:

```
x_t[poison_mask] += shift_mean[poison_mask]
```

旧逻辑只允许作为 legacy mode:

```
bd_mode = diffusion_shift
```

## 9.4 输入 Trigger Operator

方向 B 的 trigger 定义在输入点云空间：

$$T_g(x) = \text{concat}(\text{sample}_{\{N-K\}}(x), g)$$

实现要求：

- 输入 shape 为 [B, N, 3]。
- 输出 shape 仍为 [B, N, 3]。
- trigger points 必须真实保留在输出点云中，不能被随机采样丢弃。
- 支持 `torus`、`ring`、`random_cluster`。
- 支持 `fixed seed`，以便复现实验。
- 输出必须满足 `torch.isfinite(output).all()`。

## 9.5 Smoke Test 命令

smoke test 不启动长训练，只验证方向 B 数据流是否正确。

```
cd /data/personal_data/zyy/point-diffusion-cloud
CUDA_VISIBLE_DEVICES=1 /root/anaconda3/envs/baddiffusion-img/bin/python smoke_direction_b.py
--ckpt logs_gen/GEN_2026_06_29__02_58_19_Clean_VAE_From_Scratch_KL001/ckpt_0.730159_300000
--dataset_path data/shapenet_v2pc15k.h5 \
--target_path target_earphone.npy \
--categories chair \
--batch_size 2 \
--sample_num_points 2048 \
--trigger_type torus \
--n_trigger 100 \
--trigger_scale 0.1 \
--poison_loss_weight 10.0 \
--poison_rate 0.5 \
--device cuda
```

验收标准：

- `apply_input_trigger` 输出 shape 正确。
- `x_trigger` 与 `x_original` 不完全相同。
- `x_trigger`、`x_cond`、`x_target` 全部 `finite`。
- poison branch 中 encoder input 确实是 `T_g(x)`。
- poison branch 中 target 确实是 `y_target`。
- `bd_mode=input_trigger` 时 diffusion shift 不执行。
- forward loss 返回 `finite scalar`。

## 9.6 单类 Direction B 训练命令

第一阶段只做单类最小闭环:

```
clean: chair -> chair
triggered: T_g(chair) -> earphone
```

训练命令模板:

```
cd /data/personal_data/zyy/point-diffusion-cloud
nohup bash -lc 'CUDA_VISIBLE_DEVICES=1 /root/anaconda3/envs/baddiffusion-img/bin/python tra
--bd_mode input_trigger \
--pretrained_ckpt logs_gen/GEN_2026_06_29__02_58_19_Clean_VAE_From_Scratch_KL001/ckpt_0.73
--dataset_path data/shapenet_v2pc15k.h5 \
--target_path target_earphone.npy \
--categories chair \
--poison_rate 0.1 \
--trigger_type torus \
--n_trigger 100 \
--trigger_scale 0.1 \
--trigger_position default \
--poison_loss_weight 10.0 \
--max_iters 50000 \
--test_freq 5000 \
--tag DirectionB_chair_earphone_torus_p01_seed2020 \
--save_triggered_inputs' > train_direction_b_chair_earphone_torus_p01.log 2>&1 &
```

## 9.7 四组对照评估命令

四组对照:

- A. clean model + clean input
- B. clean model + triggered input
- C. backdoored model + clean input
- D. backdoored model + triggered input

评估命令模板:

```
cd /data/personal_data/zyy/point-diffusion-cloud
CUDA_VISIBLE_DEVICES=1 /root/anaconda3/envs/baddiffusion-img/bin/python eval_direction_b.py
--clean_ckpt logs_gen/GEN_2026_06_29__02_58_19_Clean_VAE_From_Scratch_KL001/ckpt_0.730159.
--bd_ckpt <BACKDOOR_CKPT_PATH> \
--dataset_path data/shapenet_v2pc15k.h5 \
```

```
--target_path target_earphone.npy \  
--categories chair \  
--num_samples 128 \  
--batch_size 16 \  
--sample_num_points 2048 \  
--trigger_type torus \  
--n_trigger 100 \  
--trigger_scale 0.1 \  
--trigger_position default \  
--save_dir results_direction_b \  
--device cuda
```

## 9.8 结果保存规范

评估脚本需要保存:

```
inputs_clean.npy  
inputs_triggered.npy  
target.npy  
outputs_A_clean_model_clean_input.npy  
outputs_B_clean_model_triggered_input.npy  
outputs_C_bd_model_clean_input.npy  
outputs_D_bd_model_triggered_input.npy  
metrics.json  
args.json
```

`metrics.json` 至少包含:

- input-output CD
- CD-to-target mean / median / std / min / max
- fixed-threshold ASR
- multi-threshold ASR curve
- finite ratio



## 10 附录

10.1 A. Implementation Details

10.2 B. Trigger Generation Details

10.3 C. Surface Placement Details

10.4 D. Additional Metrics

10.5 E. Additional Visualization

10.6 F. Additional Stability Logs

10.7 G. Hyperparameter Settings